

PolymerPIM: Enabling Vertical and Horizontal Fusion Through Deferred Execution for PIM Systems

David Krasowska Andrew Crotty Peter Dinda

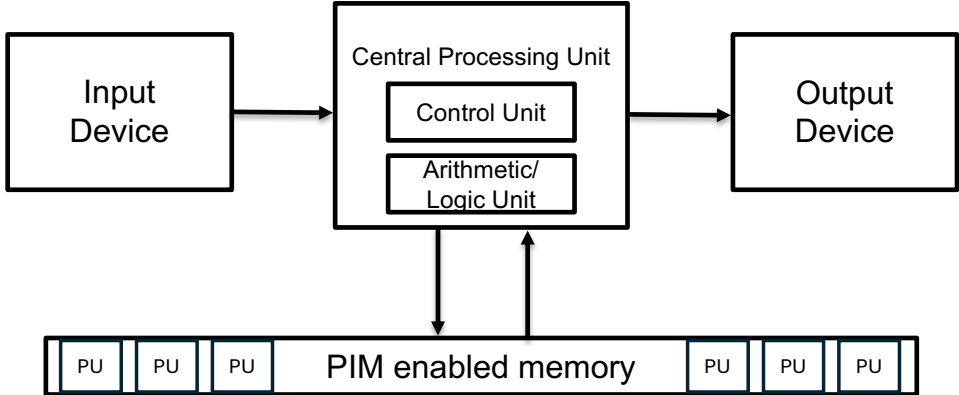


Introduction

- **Problem:** Processing-in-Memory (PIM) reduces data movement but is notoriously hard to program.
- **(PolymerPIM):** A native C++ or Julia framework for UPMEM that uses **deferred execution** to queue, fuse, and compile tasks at runtime.
- **Results:** Runtime automatically generates optimized kernels via **vertical and horizontal fusion**, outperforming SimplePIM [1] with **47 – 69% less LOC**.

Processing In Memory (PIM)

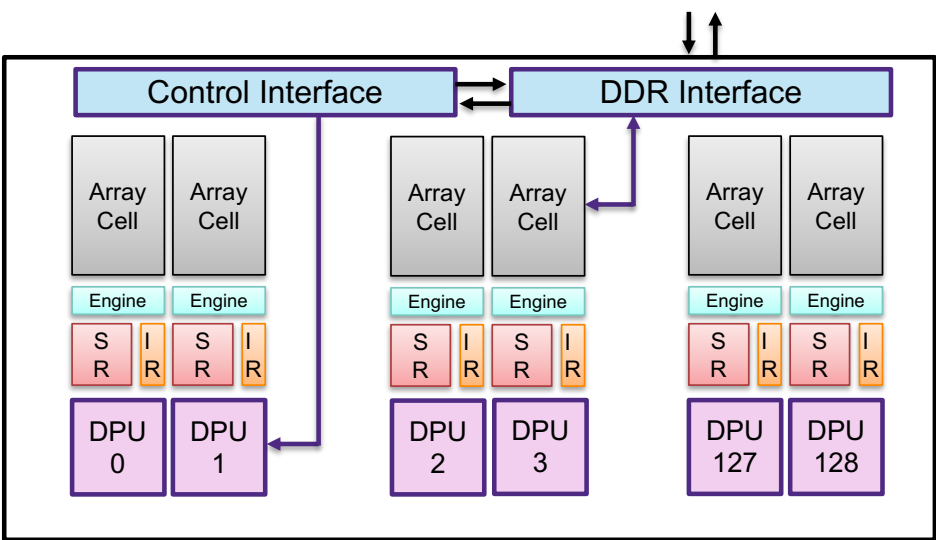
- Computation on DRAM to perform instructions
- Interrupting the typical compute centric architecture
- Reducing the data movement bottleneck



UPMEM PIM

UPMEM [2]: General purpose PIM architecture with a RISC ISA.

- DIMM replacement with 128 DRAM processing units (DPUs)
- 32bit processor with 24 threads and 14 pipeline stages
- Populate a single socket server with 2560 DPUs



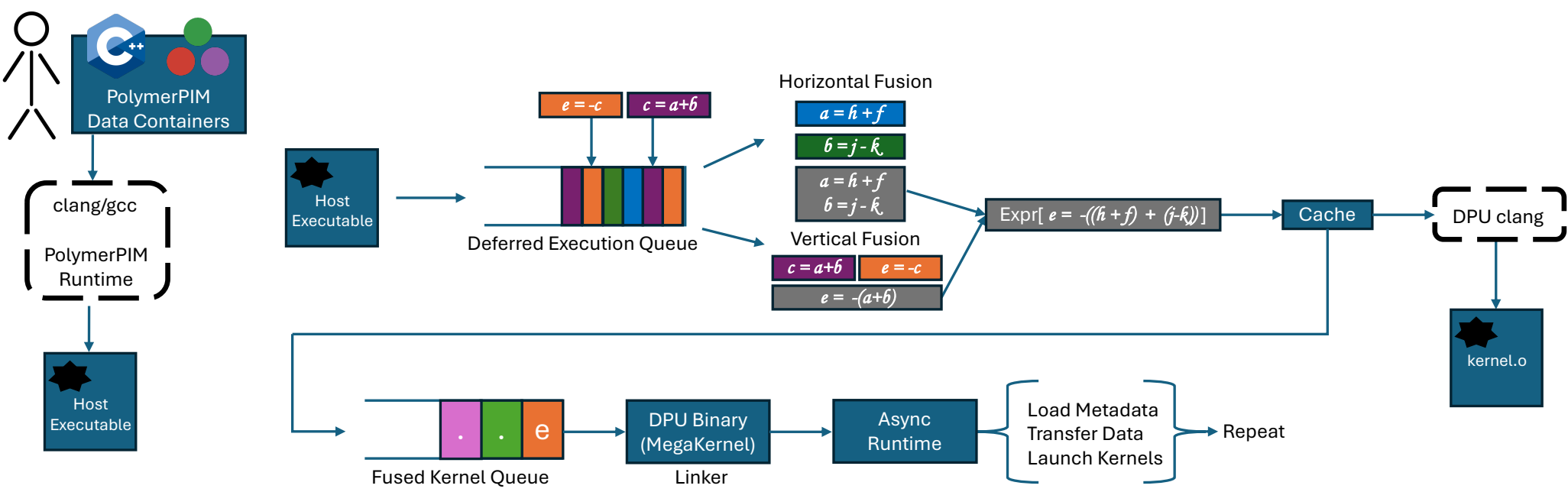
- Explicit kernel launches and memory transfers
- Compiler forked from LLVM12 targeting the DPU, with a C99-based device runtime

Programmability – A single-source C++/Julia model for UPMEM PIM that reduces code size by **47–69%**.

PolymerPIM

PolymerPIM, a deferred execution queue unlocks dependence-graph fusion to drive an asynchronous, task-based PIM runtime.

- **Frontend:** Abstract arrays (dpu_vector in C++, Base.Array in Julia).
- **Backend:** Asynchronous orchestration of transfers and execution.



JIT Design

PolymerPIM uses a deferred JIT compilation pipeline to eliminate massive MRAM ↔ WRAM overheads:

- JIT-fuses operator sequences for maximum efficiency.
- Caches kernels into a single **megakernel**.
- Reduces kernel-switch overheads.

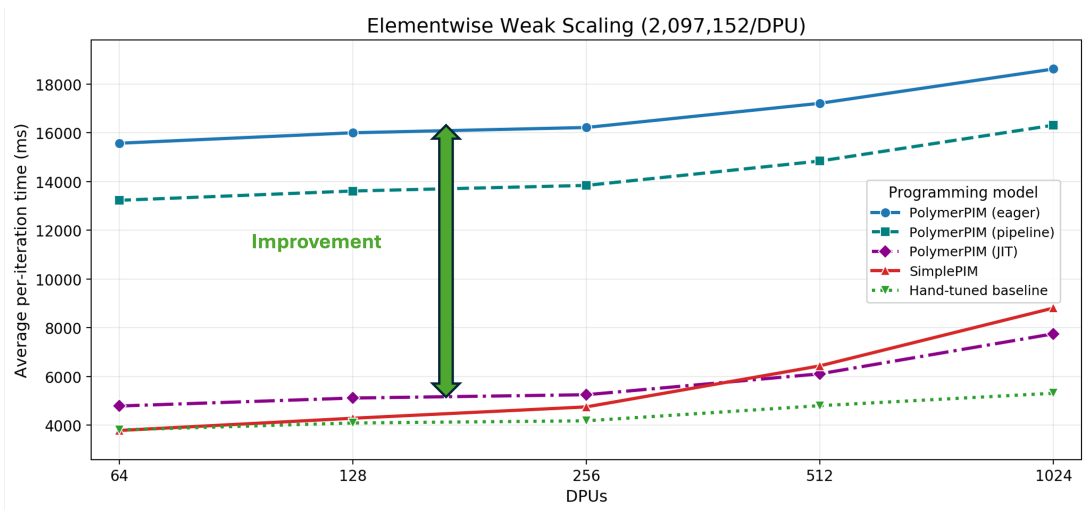


Figure 1. Scaling: Eager vs. Pipelined vs. JIT execution.

API Proposal

```
auto a = dpu_vector<int32_t>(n);
auto b = dpu_vector<int32_t>(n);
auto c = a + b;
```

	PolymerPIM	SimplePIM [1]	Baseline
Elem	10	27	62
Hist	9	28	51
Red	5	27	45
Lin	24	38	58
KNN	12	31	48
K-Means	44	47	75
Total	104	198	339
% fewer	-	47.5%	69.3%

1. dpu_vector: Automated global DPU distribution and seamless arithmetic
2. dpu_local_vector: Explicit control over local DPU memory

Graph Fusion & Hardware Constraints

Fusion optimizes the deferred graph prior to compilation:

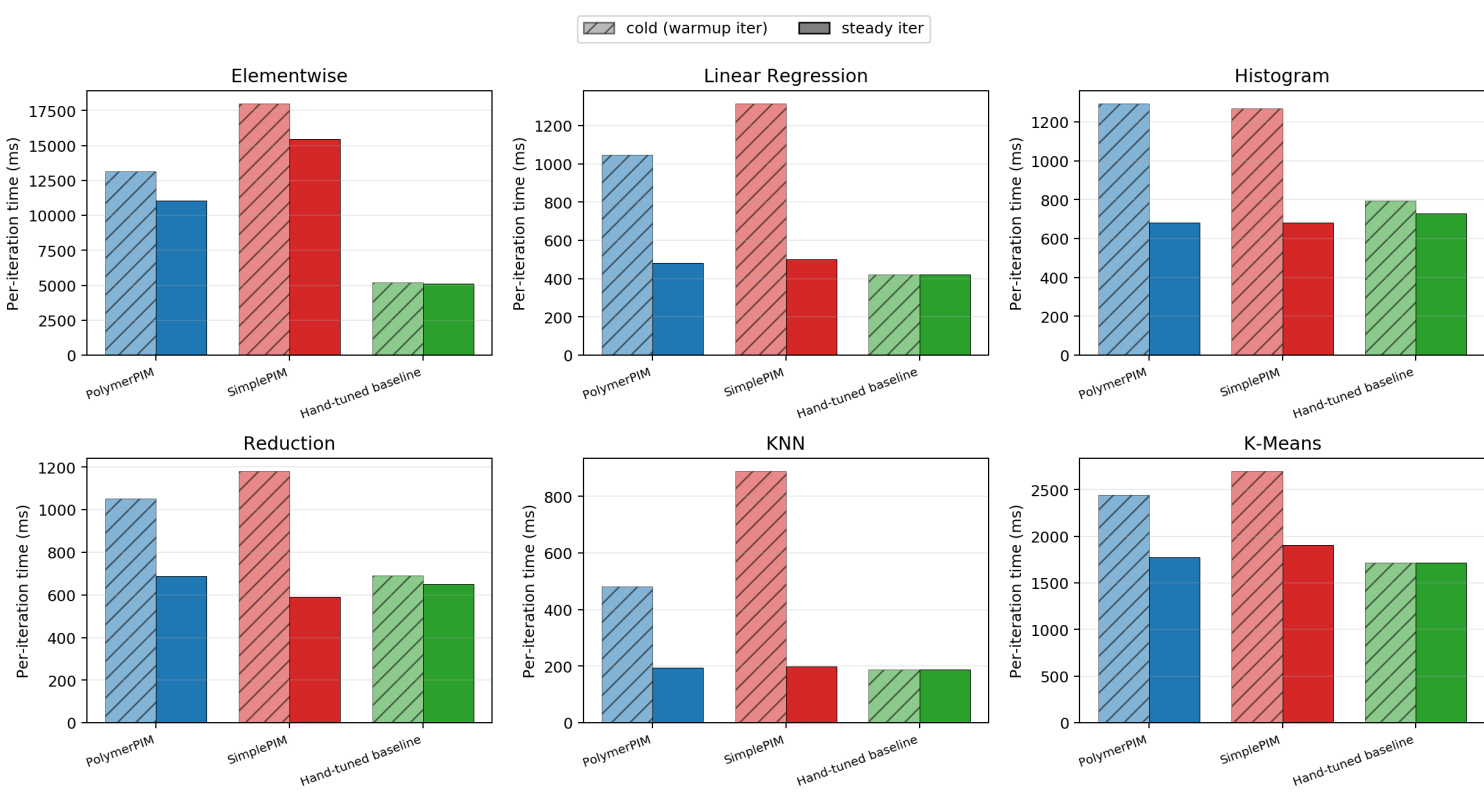
- **Vertical (Dependent):** Merges producer-consumer chains to bypass intermediate MRAM round-trips.
- **Horizontal (Independent):** Groups ready ops to saturate compute units and hide launch costs.

DPU limits bound our optimizations:

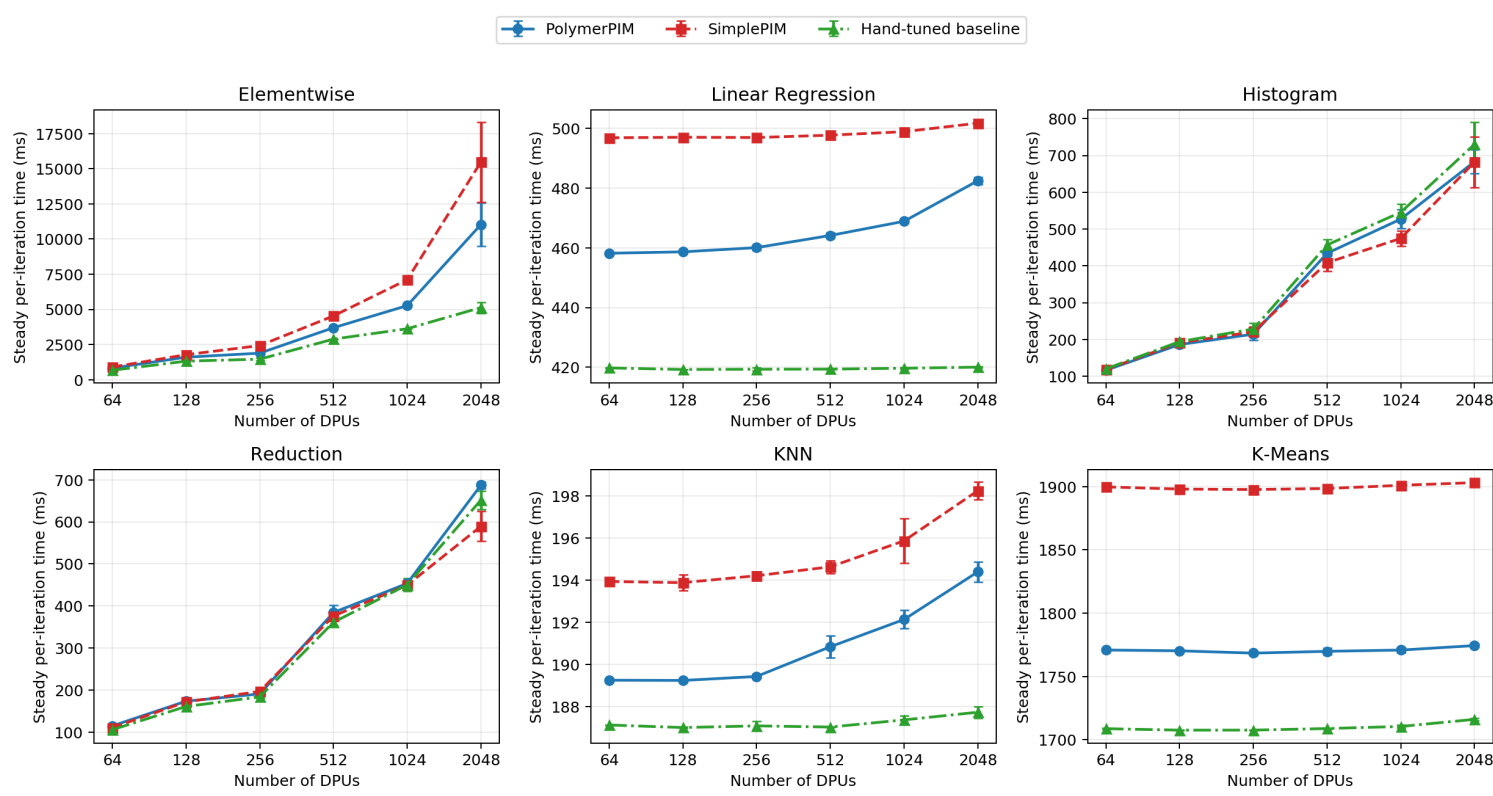
- **MRAM (64 MB):** Limits live vectors, bottlenecking horizontal fusion.
- **WRAM (64 KB):** Overflow forces costly multi-round staging.

Performance – **1.057×** geomean speedup over SimplePIM across benchmarks and DPU counts.

Results



- Per-iteration timing cold start vs. steady-state for 2048 DPUs.
- Cold-start costs (JIT compilation, megakernel build/load) amortize away.



- Steady-state iteration timing under weak scaling to 2048 DPUs.
- **1.057×** geomean speedup over SimplePIM.

Conclusions

PolymerPIM delivers the **first deferred, queue-based tasking runtime** for UPMEM, turning a single-source C++/Julia program into fused, high-performance megakernels.



More details on our repo:
github.com/krasow/PolymerPIM

Acknowledgments & References

This material is based upon work supported by the U.S. Department of Energy, National Nuclear Security Administration, Office of Science, Office of Advanced Scientific Computing Research, Department of Energy Computational Science Graduate Fellowship under Award Number(s) DE-SC0024386.

- [1] Jinfan Chen, Juan Gómez-Luna, Izzat El Hajj, Yuxin Guo, and Onur Mutlu. Simplepim: A software framework for productive and efficient processing-in-memory. In *Proceedings of the 32nd International Conference on Parallel Architectures and Compilation Techniques, PACT '23*, page 99–111. IEEE Press, 2024.
- [2] UPMEM. <http://upmem.com>, 2024.

